

Trabalho Prático 3

Consultas ao Sistema Logístico

Júlia Almeida Luna (202401424)

julialuna@ufmg.br

1. Introdução

Os Armazéns de Hanoi possuem uma nova demanda, agora desejam uma nova funcionalidade ao Sistema Logístico: consultas em tempo real sobre o histórico de movimentações. Há dois tipos de consultas a serem feitos:

Consulta por pacote: dado o identificador de um pacote apresentar os eventos associados a ele até o momento da consulta.

Consulta por cliente: dado o nome de um cliente apresentar uma lista com o primeiro e último evento de pacotes que o cliente esteja associado como remetente ou destinatário.

Com o propósito de realizar estas consultas, guardou-se um vetor único de todos os eventos em ordem cronológica (evitando duplicação de dados) e utilizou-se índices relacionados a pacotes e clientes guardados em árvores balanceadas que garantiram o processamento de eventos e consultas eficientes.

2. Método

O Sistema de Consultas Logísticas foi implementado em C++ utilizando programação orientada a objetos. Seus principais módulos são o Armazenador de Eventos (*EventStorage*), Armazenador de Pacotes (*PackageStorage*), Armazenador Clientes (*ClientsStorage*), além de um Executor de Consultas (*QueriesExecutor*). Estas estruturas principais utilizam de estruturas de dados clássicas que asseguram organização modular e alto desempenho nas operações de inserção e busca, que são descritas a seguir:

- Lista: esta estrutura é utilizada para guardar identificadores de pacotes relacionados a um cliente e o armazenamento de referências aos eventos correspondentes a um pacote no *EventStorage*. A vantagem em seu uso consiste principalmente na alocação flexível de memória em tempo de execução, permitindo que a lista cresça ou encolha conforme o volume de dados, sem necessidade de tamanho fixo pré-definido.
- Árvore AVL: a árvore binária de busca auto balanceada é utilizada em *PackageStorage* e em *ClientsStorage* para o armazenamento de informações de clientes e de pacotes. A árvore AVL se mostra uma ferramenta poderosa no sistema de consultas graças às rotações que mantêm sua altura em $\log(n)$, o que assegura que operações de buscas, inserções e atualizações ocorram em tempo $O(\log(n))$.

2.1. Armazenador de Eventos

O *EventStorage* gerencia o registro de todos os eventos em um vetor dinâmico, cujo tamanho inicial é crescente. Como o número total de eventos não é conhecido de antemão, cada chamada a *'addEvent'* verifica se há espaço livre; caso contrário, ela invoca *'increaseStorage'*, duplicando a capacidade do vetor. Esta função (*'addEvent'*) é chamada a cada evento lido durante a leitura do arquivo.

Essa estratégia de redimensionamento e uso de array garante que o acesso a qualquer evento por índice seja sempre feito em tempo $O(1)$, o que mantém os custos das consultas baixos e previsíveis.

É importante ressaltar também que cada evento possui uma chave contendo o tempo que ele ocorreu e o identificador do pacote relacionado a ele. Esta chave é essencial para que os eventos dos resultados das consultas sejam impressos ordenados.

2.2. Armazenador de Pacotes

O Armazenador de pacotes possui uma árvore balanceada em que cada nó possui o identificador do Pacote (usado como chave), índices do array de *EventStorage* correspondentes ao primeiro e último evento do pacote e uma lista da Struct *'PackageEvent'* que guarda o tempo que o evento

ocorreu e o índice que ele se encontra no array de *EventStorage*. Uma função essencial desse TAD é a função ‘*addEventToPackage*’, que, ao chegar a um novo evento, simplesmente anexa uma nova entrada à lista de *PackageEvent* e atualiza o índice do último evento .

Esse design, em que o índice do primeiro e último eventos (itens usados nas consultas de clientes) ficam diretamente no nó do pacote, simplifica e agiliza as atualizações: sempre que um evento novo ocorre, basta tocar um único ponto na árvore de pacotes. Se esses índices estivessem apenas nas listas ligadas aos clientes (*ClientsStorage*), seria necessário percorrer toda a árvore de clientes, localizar cada ocorrência daquele pacote e ajustar múltiplos registros, o que elevaria o custo de manutenção dos índices.

Outra função importante é ‘*getPackagesEvent*’ que localiza o pacote na árvore AVL e retorna sua lista de eventos.

2.3. Armazenador de Clientes

O Armazenador de Clientes, assim como o de pacotes, possui uma árvore cujos nós possuem o nome do Cliente (usado como chave) e uma lista de inteiros que possuem os identificadores dos pacotes em que o cliente foi remetente ou destinatário.

Um método essencial neste TAD é o ‘*addPackageToClient*’ que localiza o cliente na árvore balanceada e adiciona o identificador do pacote à sua lista, estabelecendo a relação cliente pacote.

Já a função ‘*getPackages*’ faz a busca pelo nó referente ao cliente na árvore e retorna os identificadores de seus pacotes, este processo é essencial para as consultas de clientes(CL) de forma eficiente.

O ponto principal dos armazenadores é que nenhum evento é copiado ou duplicado: mantêm-se apenas referências (índices ou listas de IDs) que apontam para posições no vetor único do *EventStorage*.

Conforme ilustra o Diagrama 1 abaixo, cada nó da Árvore AVL de Pacotes armazena os índices *startEventId* e *endEventId*, que delimitam o segmento de eventos daquele pacote no vetor cronológico. Do mesmo modo, cada nó da Árvore AVL de Clientes guarda uma lista de IDs de pacotes, e cada um desses IDs remete ao mesmo segmento de eventos em *EventStorage*.

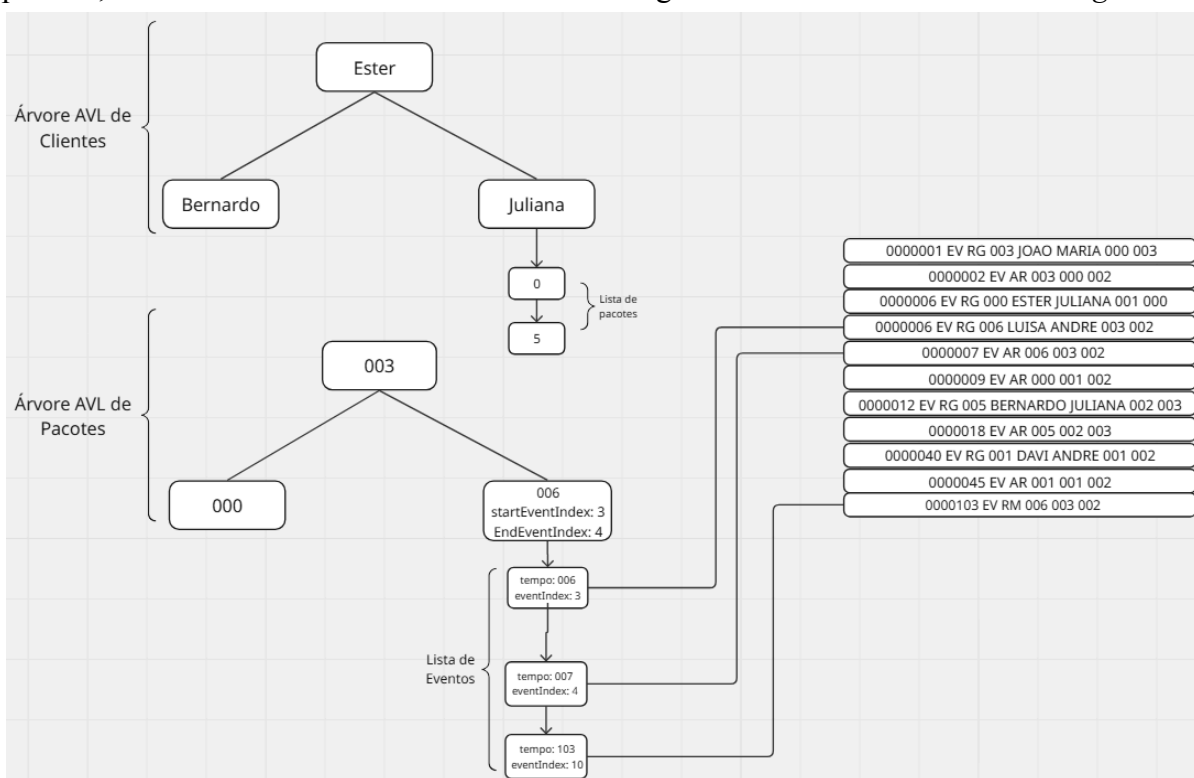


Figura 1: Conexão entre ClientsStorage, PackageStorage e EventStorage. As setas mostram como clientes e pacotes referenciam, sem duplicar, os eventos armazenados no vetor único.

2.4. Executor de Consultas

O Executor de Consultas armazena, em um vetor de objetos do tipo *Query* (struct que contém todas as informações da consulta), todas as consultas lidas do arquivo de entrada. Com base nesse vetor, seu método principal, *executeAllQueries*, processa cada consulta considerando apenas os eventos que já ocorreram até o momento em que ela foi emitida.

Antes de detalhar *executeAllQueries*, vale apresentar suas duas funções auxiliares:

A função *executeClientQueries* recebe o nome de um cliente, solicita ao *ClientsStorage* a lista de IDs de pacotes associados a esse cliente e, para cada pacote, requisita ao *PackageStorage* os índices de início e fim do seu histórico no *EventStorage*. Em seguida, reúne todos os eventos correspondentes e os ordena por uma chave composta (timestamp do evento + ID do pacote), garantindo que o resultado seja impresso em ordem cronológica exata.

A função *executePackageQueries* executa as consultas de um pacote através de seu identificador, e assim solicita a *PackageStorage* os eventos relacionados ao pacote. A partir dos índices que são retornados pela função, é possível acessar os eventos em *EventStorage* e imprimi-los como resposta a consulta.

Um ponto importante das consultas é que elas devem retornar um resultado condizente com os eventos ocorridos até a execução da consulta. Assim, cada *Query* armazena o índice de *EventStorage* correspondente ao último evento anterior à própria consulta.

O Executor de Consultas processa cada consulta em duas etapas, sem jamais voltar a eventos já consumidos. Na primeira etapa, ele avança no vetor de eventos a partir do ponto em que parou na consulta anterior até alcançar o índice registrado na *Query* atual. Para cada evento encontrado, se for do tipo “RG” (registro de geração) ele insere o novo pacote e o cliente nos seus armazenadores e estabelece o vínculo cliente pacote (adicionando o ID do pacote na lista de pacotes do cliente); caso contrário, chama *addEventToPackage* para anexar o evento a lista de eventos do pacote e atualizar o índice final desse pacote em *PackageStorage*. Ao término desse avanço, todos os índices — de eventos, de pacotes e de clientes — refletem com exatidão o estado do sistema imediatamente antes da emissão da consulta.

A segunda parte consiste em verificar o tipo da consulta, caso ela seja de pacote, executa-se a função *executePackageQueries*, caso seja de cliente, executa-se a função *executeClientQueries*.

Na consulta subsequente, o processamento de eventos retoma exatamente do ponto em que parou, assegurando que cada evento seja “executado” apenas uma vez e mantendo o método *executeAllQueries* eficiente.

3. Análise de Complexidade

Nesta seção, será apresentada a análise de complexidade não apenas do mecanismo principal das consultas de eventos, mas também de suas funções auxiliares fundamentais e as inicializações dos armazenamentos que são fundamentais para a execução das consultas.

3.1. Leitura e armazenamento de eventos e consultas

O sistema se inicia com uma leitura única do arquivo (que contém N eventos e Q consultas). Como cada linha é lida apenas uma vez e gera um armazenamento que tem custo constante, temos que a complexidade de tempo desta etapa é $O(N+Q)$.

Além disso, como guardamos cada evento e cada consulta temos que a complexidade de espaço desta etapa é $O(N+Q)$

3.2. Inicialização do Sistema

Além da leitura única do arquivo de entrada, que traz tanto os eventos quanto às consultas, cada evento anterior a uma consulta deve ser “executado” para atualizar os armazenamentos e refletir os efeitos dos eventos. Dependendo do tipo de evento, duas rotinas distintas são acionadas:

Caso o evento seja de registro, 3 ações podem ser tomadas:

- Adicionar cliente (método ‘*addClient*’): insere o nó do cliente na AVL de clientes, em custo $O(\log(C))$, em que C é o número de clientes.
- Adicionar pacote (função ‘*addPackage*’): insere o nó do pacote na AVL de pacotes, em $O(\log(P))$, em que P é o número de pacotes.
- Vincular pacote ao cliente (método ‘*addPackageToClient*’): localiza o cliente na AVL $O(\log(C))$ e anexa o ID do pacote ao final de sua lista $O(1)$, totalizando $O(\log(C))$

Caso o evento não seja de registro, apenas atualizamos seu histórico através da função ‘*addEventToPackage*’ que consiste em achar o nó na árvore AVL correspondente ao pacote e adicionar o evento ao fim da lista de eventos, essas ações têm custo $O(\log(P))$ e $O(1)$ respectivamente, logo o custo total é $O(\log(P) + 1) = O(\log(P))$.

Considerando que há N eventos no total, dos quais X são de registro RG e $N-X$ são outros tipos, o custo de executar todos os eventos é dado pela soma:

$$X \times (O(\log(C)) + O(\log(P))) + (N-X) \times O(\log(P)) = O(X(\log(C) + \log(P)) + (N-X)\log(P)).$$

Sobre a complexidade de espaço, em *PackageStorage* guardamos um nó para cada pacote e uma referências a seus eventos, como são N eventos no total, temos uma complexidade de $O(P+N)$, como $N > P$, temos $O(N)$. Em *ClientsStorage* temos um Nó para cada cliente, ou seja, C nós. Como um pacote tem um remetente e um destinatário, ele gera duas relações em dois clientes, assim temos um total de $2P$ pacotes nas listas de clientes. Logo, em *ClientsStorage* temos $O(2P+C)$.

No total, tendo em vista que $N > P$ e $N > C$, o processo de inicialização do sistema tem uma complexidade de espaço:

$$O(N + 2P + C) = O(N)$$

3.3. Consulta por pacote

O processo de consulta por pacote consiste na busca pelo nó referente ao pacote na árvore AVL cujo custo é $O(\log(P))$. Após isso, a lista dos índices de eventos referentes ao pacotes é retornada e como o custo de acessar um evento em *EventStorage* através de seu índice é $O(1)$, considerando E o número de eventos de um pacote, temos o custo $O(E)$.

Como é esperado um número considerável de pacotes (maior que o número de eventos por pacote), temos que a consulta por pacote é bem eficiente e possui uma complexidade de tempo $O(\log(P) + E) = O(\log(P))$. Como nenhuma memória adicional é utilizada neste processo, temos que sua complexidade de espaço é $O(1)$.

3.4. Consulta por cliente

O processo de consulta por clientes consiste na busca pelo nó referente ao cliente na árvore balanceada e o retorno da lista de identificadores de pacotes relacionados ao cliente, essa busca possui uma complexidade de $O(\log(C))$.

Sendo T o número de pacotes relacionados ao clientes, fazemos T buscas na árvore AVL de pacotes, que por um custo constante retorna o índice do primeiro e o último evento, como o

acesso a um evento através de seu índice em *EventsStorage* é constante, temos um custo de $O(T * \log(P))$.

Importante ressaltar que um array de eventos é criado para a impressão ordenada deles, como temos T pacotes cada um com dois eventos, este array tem tamanho $2T$. Devido ao fato de os eventos estarem ordenados no arquivo de entrada, temos que este array possui poucas ou nenhuma desordenação, sendo assim, ao ordená-lo usando o algoritmo de ordenação *InsertionSort*, temos um custo $O(2T) = O(T)$ para ordenação.

A complexidade de tempo se torna então:

$$O(\log(C) + T * \log(P) + T) = O(\log(C) + T * \log(P))$$

Como um array de eventos de tamanho $2T$ é criado para a impressão ordenada deles, a complexidade de espaço se torna $O(2T) = O(T)$.

É importante ressaltar que o uso de índices na implementação combinado ao armazenamento em árvores balanceadas consegue trazer uma grande eficiência a todo sistema ao diminuir a complexidade de espaço nas consultas e garantir buscas com um custo logarítmico.

3.5. Execução de todas as consultas

A execução de todas as consultas (cujo responsável é o método *executeAllQueries*) depende tanto da *inicialização do sistema* quanto da realização de consultas, desse modo, sua complexidade é a soma da complexidade destas etapas. Sendo R o número de *consultas de clientes* e K o *número de consultas de pacotes*, temos:

$$O(X(\log(C) + \log(P)) + (N - X)\log(P) + K * \log(P) + R * (\log(C) + T * \log(P)))$$

Já a complexidade de espaço é consiste na memória extra utilizada pelas duas etapas:

$$O(N + T)$$

4. Estratégias de Robustez

A implementação do Sistema de Consultas Logísticas adota diversas estratégias para garantir robustez e confiabilidade ao código. Inicialmente, tem-se a verificação de erro no acesso a arquivos, com mensagens claras indicando se o erro se relaciona ao nome ou falha de abertura. Além disso, no gerenciamento dos arrays dinâmicos de 'Events' e 'Query' durante o aumento da capacidade do array, caso a memória nova não consiga ser alocada, uma mensagem de erro é lançada e a ação é interrompida.

Ações inválidas no sistema de consultas também geram mensagens de erro especializadas. Exemplificações dessas situações são leitura de consultas ou eventos inválidos e atualizações em pacotes inexistentes.

As estruturas de dados básicas como lista e Árvore AVL contam igualmente com estratégias que garantem segurança. Por exemplo, caso um item já existente na árvore seja adicionado, a ação é interrompida e uma mensagem de erro é lançada. Já na lista, qualquer ação inválida (como índice fora de alcance ou estrutura vazia) também é interrompida e mensagens de erro são lançadas.

A implementação contou também com um gerenciamento cuidadoso da memória, o que foi possível através da utilização da ferramenta Valgrind, utilizada para detectar possíveis vazamentos de memória.

5. Análise Experimental

Para avaliar o desempenho do Sistema de Consultas em relação ao Sistema Logístico, executamos experimentos projetados para medir como diferentes parâmetros afetam o tempo de resposta. Definiu-se inicialmente um cenário de referência no qual o ambiente contava com 10 armazéns interconectados, capacidade de transporte de 5 pacotes, custo de transporte de 20 unidades e intervalo de envio de 100 unidades de tempo. Nesse cenário, foram utilizados 200 pacotes e 10 clientes, e a carga de consultas consistiu em 10 pedidos de histórico de clientes e 10 de histórico de pacotes. Esse caso base serve como ponto de comparação para as variações subsequentes em cada dimensão analisada.

5.1. Variação do Número de Pacotes

Para avaliar de forma precisa o impacto exclusivo do número de pacotes no tempo de execução das consultas, mantivemos todos os demais parâmetros do caso base estritamente inalterados, principalmente o número de consultas, de modo a ser possível identificar a influência da quantidade de pacotes no tempo de execução. Variou-se apenas o volume de pacotes nos cenários de 200, 400, 600, 800 e 950, de modo que qualquer mudança no desempenho pudesse ser atribuída unicamente a esse fator. A figura a seguir apresenta os resultados obtidos para cada uma dessas configurações.

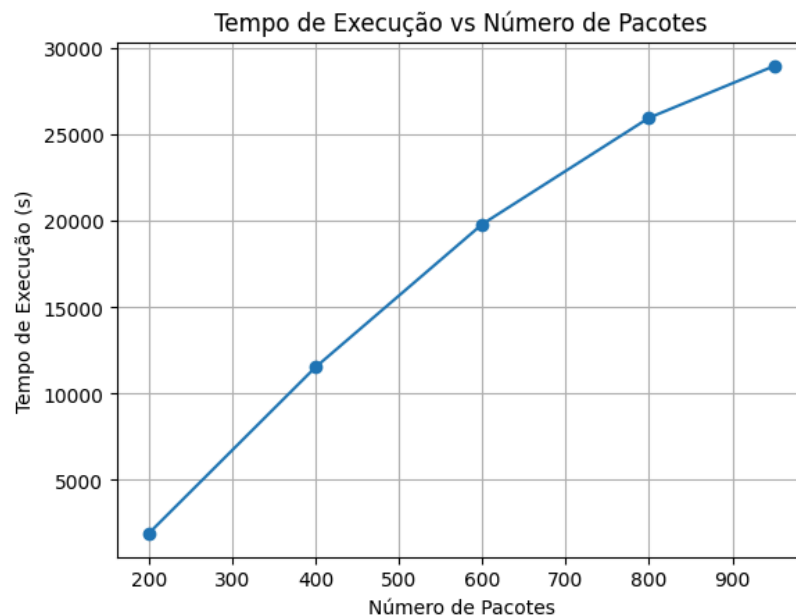


Figura 2: resultados dos tempos de execução ao variar número de pacotes

Percebe-se que, à medida que o número de pacotes cresce, o tempo de resposta das consultas também aumenta. Isso ocorre porque a complexidade tanto da fase de inicialização, em que construímos índices para todos os pacotes, quanto do próprio processamento das consultas dependem diretamente de $\log(P)$. Desse modo, quanto maior a quantidade de pacotes, maior o custo

5.2. Variação do Número de Clientes

O segundo experimento abordou a influência do número de clientes no tempo de execução de consultas. Novamente, todos os demais parâmetros foram mantidos e apenas o número de clientes foi variado entre: 10, 40, 60, 80, 100 e 120. A figura a seguir apresenta uma análise dos resultados.

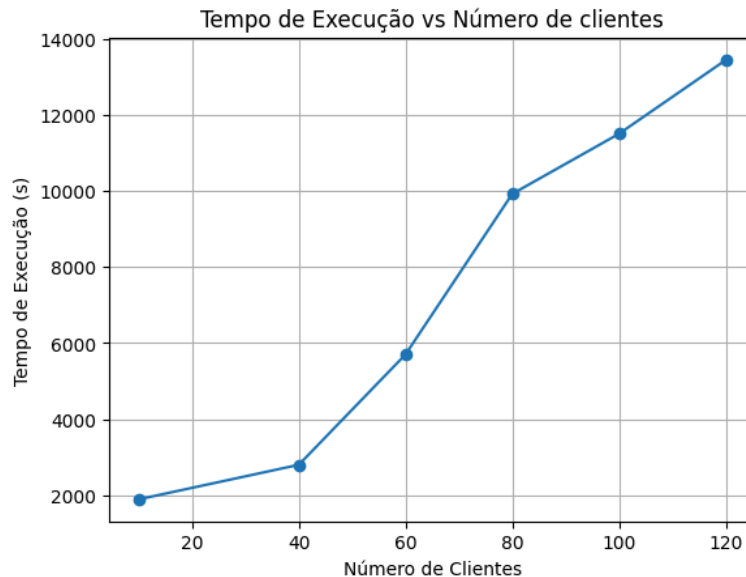


Figura 3: resultados dos tempos de execução ao variar número de clientes

Observa-se que, assim como ao aumentar o número de pacotes, elevar a quantidade de clientes também provoca um acréscimo no tempo de execução das consultas. Isso ocorre porque o número de clientes influencia a complexidade tanto da fase de inicialização quanto do processamento das consultas, já que operações de busca e atualização nas estruturas de dados balanceadas dependem diretamente de $\log(C)$. Dessa forma, quanto maior o total de clientes, maior o custo computacional e, conseqüentemente, o tempo de resposta do sistema.

5.3. Variação do Número de Eventos

A fim de verificar a influência do número de eventos, foram criados cenários no Sistema Logístico que geram diferentes níveis de contenção de modo que um maior nível de contenção gera mais eventos. Para isso variou-se o número de armazéns e a capacidade por transporte, parâmetros que influenciam diretamente o número de eventos.

Foram estabelecidos cenários de baixa, média/baixa, média/alta e alta contenção, esse cenários geraram respectivamente 2071, 2265, 2932, 3789 e 4988 eventos. A figura a seguir mostra os resultados tanto do tempo de execução quanto do tempo de leitura do arquivo:

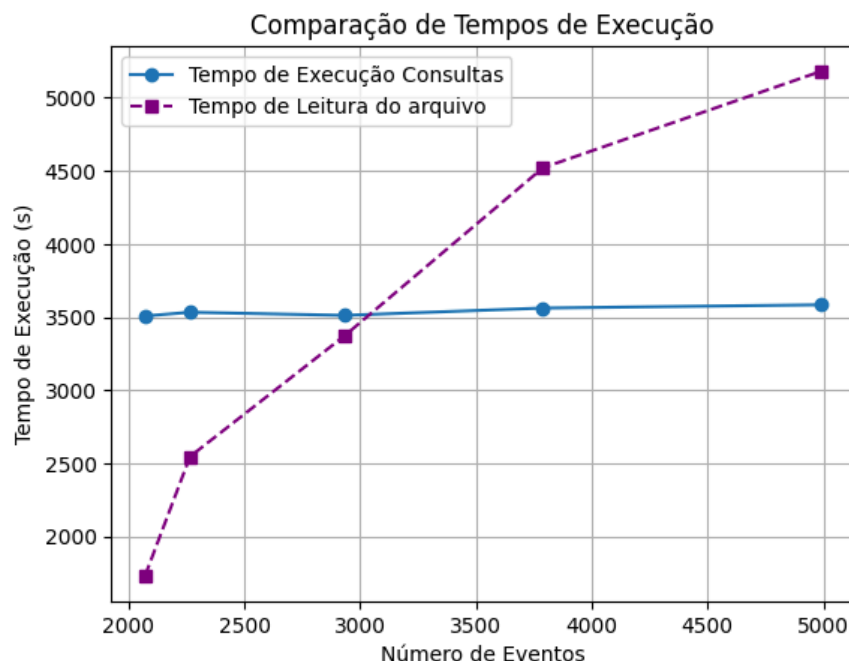


Figura 4: tempos de execução e leitura ao variar número de eventos

Nota-se que, uma vez que a complexidade das consultas estão principalmente baseadas no número de pacotes, consultas e clientes, o número de eventos não teve grande influência no tempo de execução do sistema. Entretanto, é possível verificar que o número de eventos influencia diretamente o tamanho do arquivo de entrada, que por sua vez gera um aumento no tempo de leitura da entrada.

5.4. Aumento da Carga de Trabalho (Consultas)

Por fim, foi analisado o aumento do número de consultas por pacote e por cliente entre os valores: 10, 20, 25, 30 e 40 (um parâmetro variado por vez). Uma vez que esta variação impacta diretamente na carga do trabalho do Sistema de Consultas, foi possível verificar também o aumento no tempo de execução.

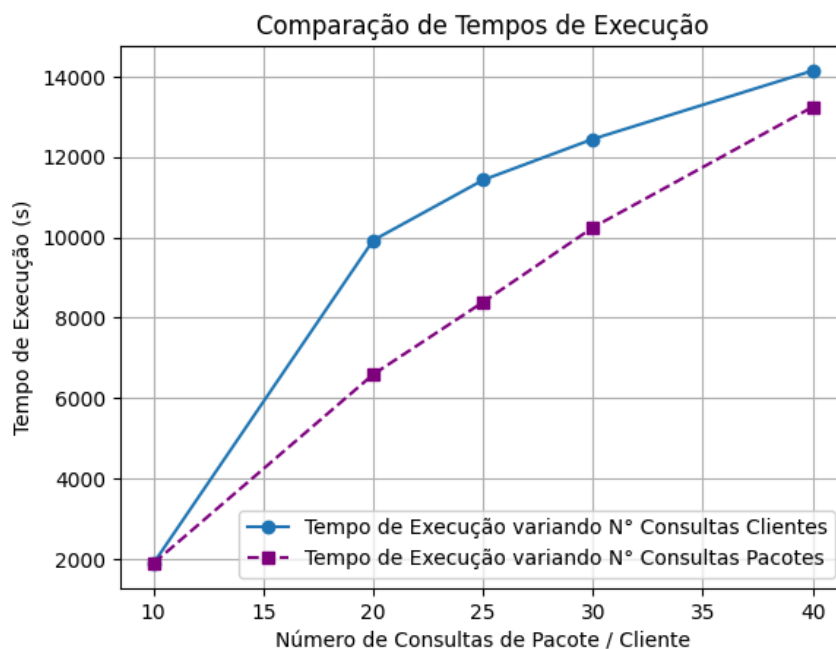


Figura 5: variação do tempo de execução com diferentes cargas de trabalho

Um ponto interessante a se analisar é que, como as consultas de cliente têm complexidade maior, $O(\log(C)+T*\log(P))$, em comparação com as consultas por pacote, cuja complexidade é apenas $O(\log(P))$, ao variar separadamente o número de consultas de cada tipo (conforme ilustrado no gráfico), fica evidente que o tempo de resposta das consultas de cliente cresce mais acentuadamente do que o das consultas por pacote.

5.5. Conclusão

A partir dos experimentos, foi possível verificar que a quantidade de clientes, pacotes e consultas que realmente interferem no tempo de execução e inicialização do sistema. Já o número de eventos, apesar de não interferir tanto no tempo de execução, exerce influência sobre o custo de leitura do arquivo.

6. Conclusões

A solução para a demanda dos Armazéns de Hanoi contou com a implementação de um sistema de consultas para o simulador logístico dos Armazéns Hanoi, integrando leitura única de eventos e comandos, índices em árvores AVL e estruturas auxiliares para responder eficientemente a consultas por pacote e por cliente.

Este trabalho prático trouxe diversos aprendizados, entre eles destaco a utilização de índices estratégicos como tática para acelerar consultas e utilizar menos memória adicional na execução. Além disso, este trabalho trouxe a oportunidade de utilizar na prática o que vimos nas aulas sobre Árvores AVLs, além de poder utilizar esta estrutura em um problema real, através dos experimentos pude notar os benefícios em utilizá-la para buscas eficientes.

Outro ponto importante sobre os experimentos, é que a partir deles pude entender melhor a detectar a existência ou não de influência no tempo de execução a partir da alteração de parâmetros.

7. Bibliografia

[1] A. Lacerda e W. Meira Jr., Slides virtuais da disciplina de Estruturas de Dados, 2020.

Disponível via Moodle.

[2] T. H. Cormen, C. E. Leiserson e R. L. Rivest, Algoritmos. 3ª ed., 2017.

[4]IME.USP. Busca em vetor ordenado. Disponível em:

<https://www.ime.usp.br/~pf/algoritmos/aulas/bubi.html>

[4] W3Schools. DSA AVL Trees. Disponível em:

https://www.w3schools.com/dsa/dsa_data_avltrees.php

8. Proposta de Aprimoramento: Consultas sobre movimentos de pacotes de um armazém dado um intervalo de tempo

O sistema de busca implementado no trabalho prático contava apenas com duas consultas: a dos eventos de pacotes relacionados à clientes e os eventos relacionados a um único pacote.

Entretanto, um novo tipo de consulta sugerido pelo enunciado foi implementado de modo a tornar o sistema de consultas ainda mais completo.

A nova consulta consiste em: dado um intervalo de tempo, retornar como resposta todas as movimentações de pacotes ocorrida em um determinado armazém.

● Método e Complexidade:

Para implementar essa nova consulta, foi criada a função

'executeWarehouseQueriesAtTimeInterval'. Ela explora o fato de que o vetor de eventos já está ordenado por timestamp, o que torna possível usar busca binária em tempo $O(\log(N))$ para localizar o primeiro evento cujo tempo seja maior ou igual ao tempo inicial do intervalo. No algoritmo, o índice *"right"* resultante da busca binária é aproveitado de modo que se não houver evento exatamente no tempo inicial, iniciamos no próximo evento disponível; caso haja, usamos o índice exato referente ao evento no tempo inicial.

O interessante da utilização da busca binária neste método é que o fato do vetor de eventos já estar ordenado possibilita o uso desta técnica que garante uma busca eficiente pelo intervalo de tempo, uma vez que seu custo é logarítmico.

A partir desse ponto, avançamos sequencialmente pelo vetor enquanto o tempo do evento não ultrapassar o tempo final, e para cada evento de movimentação de pacote, chegada (AR),

remoção (RM), rearmazenamento (UR) ou entrega (EN), verificamos se o armazém relacionado aos eventos coincide com o solicitado. Sempre que houver correspondência, o evento é impresso. Como essa varredura processa exatamente M eventos dentro do intervalo, seu custo é $O(M)$, resultando em um tempo total de $O(\log(N) + M)$. Nenhuma estrutura extra é alocada durante o procedimento, de modo que a complexidade de espaço permanece em $O(1)$.

- Entrada e saída

Para essa nova consulta, a entrada deve seguir o mesmo formato das demais: uma linha com o timestamp da consulta, o código “CA” (Consulta Armazém), o identificador do armazém e os tempos inicial e final do intervalo. Por exemplo:

0000125 CA 002 100 124.

Esta é uma consulta no tempo 125 sobre movimentações de pacotes no armazém 2 entre o tempo 100 e 124.

A saída correspondente, extraída do arquivo de teste do .zip da implementação extra, é:

000125 CA 002 100 124

Movimentacoes armazem 002

Pacote 005 removido do armazem 002 no tempo 102

Pacote 000 armazenado no armazem 002 no tempo 123

Pacote 001 entregue no armazem 002 no tempo 123

Pacote 004 entregue no armazem 002 no tempo 123

Pacote 006 entregue no armazem 002 no tempo 123

Pacote 003 armazenado no armazem 002 no tempo 124

Pacote 007 armazenado no armazem 002 no tempo 124

- Experimentos

Como a complexidade dessa consulta depende diretamente do número de eventos no arquivo de entrada, o tempo de execução foi medido mantendo o mesmo cenário-base e realizando cinco consultas de movimentação de pacotes por armazém em um intervalo de tempo. Para tal, utilizou-se exatamente aquelas quatro volumetrias de eventos da Seção 5 (2071, 2265, 2932, 3789 e 4988) e, em cada caso, registramos quanto tempo o sistema levou para processar essas cinco consultas. A análise dos resultados se encontra na figura abaixo:

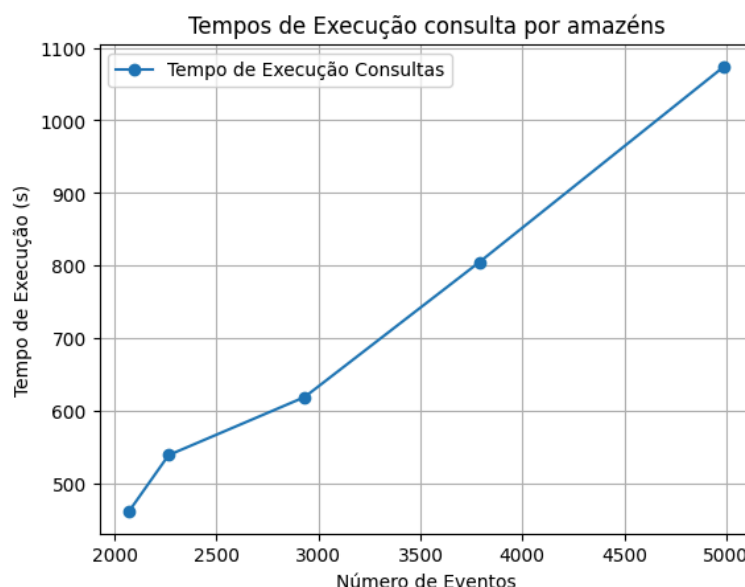


Figura 6: variação do tempo de execução com diferentes números de eventos

Nota-se que o tempo de execução dessa consulta, diferente das outras duas de pacote e clientes, cresce à medida que o número de eventos aumenta, justamente por que o $\log(\text{número de eventos})$ está presente na complexidade do método e assim interfere no tempo de execução.